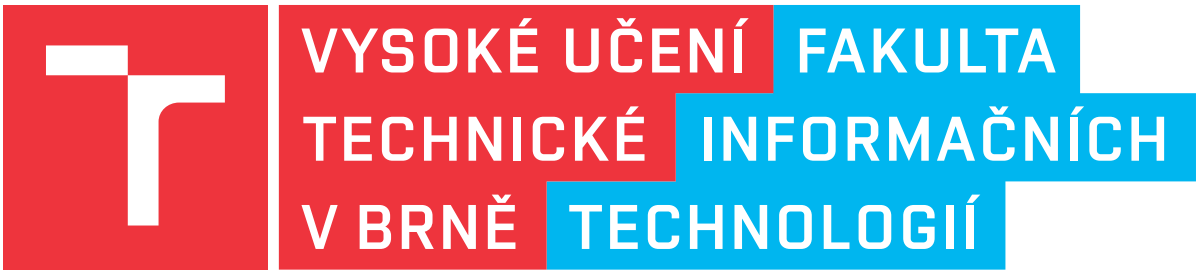

Hair simulation and rendering

Advanced Computer Graphics - Project

December 30, 2024

Bc. Peter Ďurica (xduric05)

Contents

1 Simulation and render of hair

Before hair can be properly rendered, its position must be calculated for each frame. For both simulation and rendering, special algorithms are used to secure the correct visualization of the hair strands.

1.1 Simulation

The position of the hair strands is calculated using the compute shader which is running in the background while the scene is being set up. Four textures are used to save positions of hairs for last two frames, new simulated positions in this frame and resting positions of hair.

Hair is represented as twelve segments. Each invocation of compute shader calculates its own hair strand. This means that at the end of shader, program will write new positions for each of its segments back into texture memory.

For simulating hair without wind, new position is calculated based on position from last frame, current velocity of hair, which is calculated from last 2 frames, and lastly gravity. Gravity is adjusted based on difference of time between frames and velocity is dampened by air drag which slows hair a little to always stay behind to simulate flow.

$$Pos_{i+1} = Pos_i + Vel_i * AirDrag + G * dt^2$$

Next, the position is adjusted based on three constraints. They are global, local and edge constraints. Global and local constraints adjust position based on resting position of hair and the edge constraint adjust position of segments based on their neighboring segments.

Strength of global constraint is smaller based on the distance of hair segment from its root. This makes sense since at root the hairs hold stronger then at the ends. When constraint strength for each segment is calculated, position for that segment is adjusted to resting position.

$$StrengthCon_i = StrengthCon_{i-1} - (MaxStrength/nOfHairSegments)$$

$$ConG_i = StrengthCon_i * (RestPos_i - NewPos_i)$$

Local constraint uses fixed strength of its constraint and is also influenced by next segment in hair. This makes sure that both segments move to similar directions.

$$d = RestPos_i - Pos_i$$

$$ConL_{i-1} = -0.5 * StrengthLoc * d$$

$$ConL_i = 0.5 * StrengthLoc * d$$

Finally edge constraint is integrated to make sure that each hair segment stays close to its neighbors. It calculates vector between both segments and a difference in real distance and supposed distance between segments. After that positions of segments are adjusted to better resemble supposed distance.

$$n = normalize(Pos_i - Pos_{i+1})$$

$$deltaD = length(Pos_i - Pos_{i+1}) - SegmentLength$$

$$ConE_i = -0.5 * deltaD * n$$

$$ConE_{i+1} = 0.5 * deltaD * n$$

Both local and edge constraint can be done in multiple iterations to ensure better linearity between hair segments.

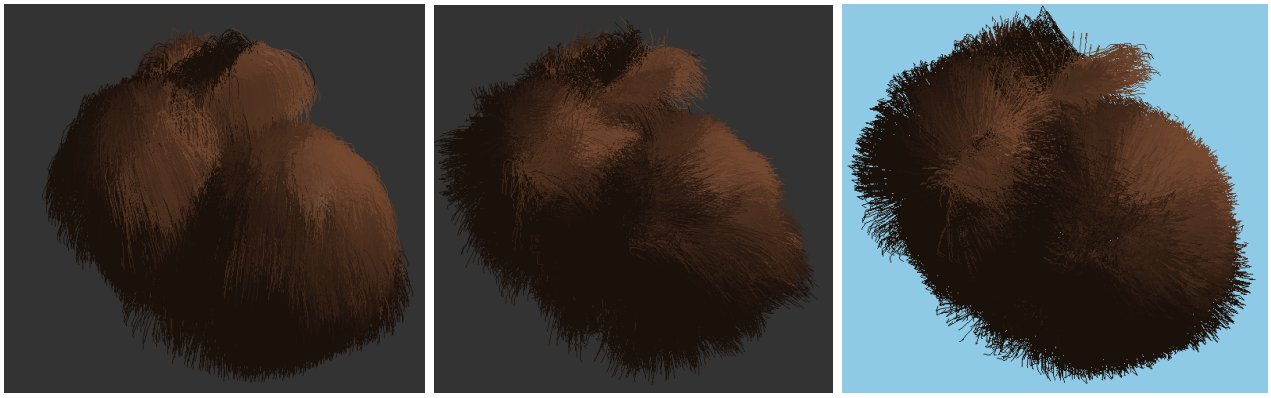

Figure 1: Simulation without wind, then with wind and lastly simulation underwater

Lastly, if wind is enabled also wind simulation is calculated. There are many different equation so I decided to not write them out and I will only describe how i calculate wind effect on hair. All equations can be found in shader's code.

First, two more wind vectors are created to simulate swirl of wind. One is directly pointing up and other is perpendicular to both the up vector and wind direction. Now with these three vectors I create 4 different wind directions that are similar to original direction to simulate turbulence. Weight is calculated for every hair segment which is used to get unique blend from 4 wind vectors. Wind force acts perpendicular to hair strands, so force is adjusted for each hair strand based on its direction and then added to previously simulated position.

If underwater rendering is turned on. It is simulated by greatly reducing gravity and increasing drag of hair. At the end all calculated new positions are saved to the texture and hair is ready to get rendered.

1.2 Rendering

After positions of master hairs in next frame are calculated, it's time to render the hairs onto the desired object. Most important parts of rendering process is tesselation and geometry shaders. Since master hairs are only generated for vertices on object, there needs to be more hairs rendered to fully cover the object. That means that more vertices need to be tessellated, from which new hairs can be rendered.

Degree of tesselation is calculated based on area of the polygon and user controlled constant. Then, in evaluation shader, new positions for tessellated vertices are calculated and passed to geometry shader.

Hair is rendered as line strip with 12 vertices. First vertex is comes from triangle on object surface and others are generated in geometry shader from master hair positions. Hair positions are passed to shaders as texture. For every tessellated vertex, a master hair is chosen that is closest to its position. When position of hair segment is pulled from texture, it's offset by small random value so that hairs don't look similar to each other. When all vertices are emitted, newly generated primitive is passed to fragment shader.

In fragment shader hair fragments are shaded using Phong. User can adjust hair color, light color and also light position. Hairs are then rendered onto the object vertices.

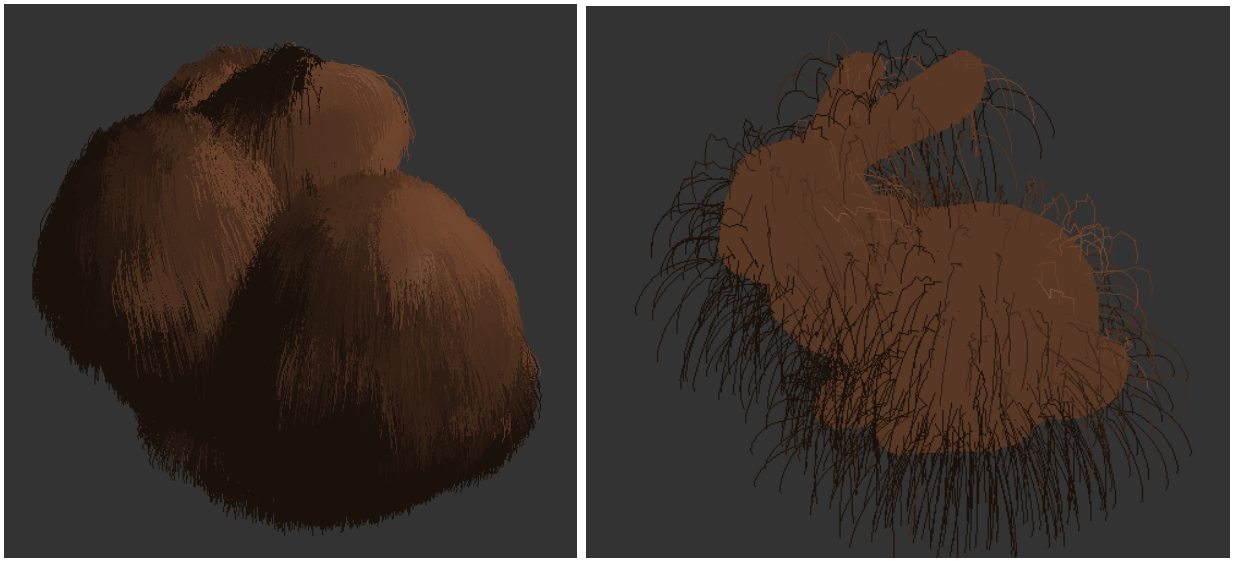

Figure 2: Difference between many tessellated vertices and none

2 Preprocessing

First part of preprocessing is creating variables for GUI and loading and compiling shaders into memory. After that demo objects are generated/loaded, which will be used to render hair on. I decided to have three objects, which are cube, sphere and Stanford bunny. The way that the hair is rendered can be achieved on any object but I concluded that these three will be enough for demonstration.

Vertices, normals and texture coordinates are generated for cube and sphere while the bunny is loaded from `FitGraphics` framework. Bunny object which is in framework doesn't have texture coordinates so i decided to just copy absolute normal values since they are in the same range and coordinates are only used to get random values from random texture.

After that it's time to generate master hairs for each vertex on object. That is done by getting position and normal for each vertex. Then new hair segments can by generated by calculating they're position from root position (vertex position) and adding normal times segment number times distance between segments.

$$hSegmentPos_i = rootPos + (i * normal * hSegmentLength)$$

When user selects an object new textures are generated and hairs are placed inside of them to be later used in rendering. In total four textures are created. First holds resting positions of master hair and it changes only when new object is selected. Next two holds hair positions for last two frames and are used in calculating new positions which are saved in the last texture. At the end of the frame these textures are swapped to be ready for calculation for next frame.

I decided to lock framerate to 60 frames per second for better stability of rendering. In render loop program first checks if object or any wind direction or light variables were changed and properly handle generating new master hairs or matrices accordingly. Then it adjusts all uniform variables inside shaders. Compute shader is dispatched first, then plain objects are rendered and only after the compute shader has finished, hair is rendered last. On exit all allocated memory is freed and textures are deleted.

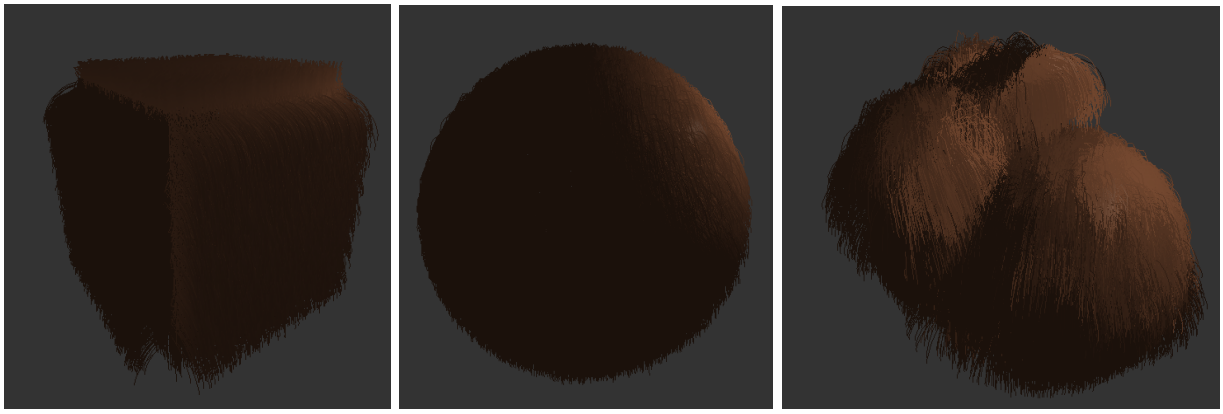

Figure 3: Demo objects in program

3 Attributes Adjustable by User

These are all attributes that users can adjust in program.

Hair

- `constraintIterations` - Set how many iterations are done when with constraints
- `densityOfTessellation` - Set how many hair gets rendered on object
- `hairColor` - Set hair color
- `hairSegmentLength` - Change length of hair segment so also adjust length of hair
- `hairStiffness` - Make hair stiffer or less stiffer
- `randomnessIntensity` - Adjust how much will random numbers change positions of hair

Light

- `lightColor` - Change color of light
- `lightPosition` - Change position of light

Object

- `object` - Select object
- `objectColor` - Change color of underlying object
- `viewObject` - Set visibility of underlying object

Underwater

- `enable` - Set underwater rendering

Wind

- `enableWind` - Enable or disable wind
- `direction` - Change direction of wind
- `strength` - Set strength of wind
- `constantWind` - Set always maximum wind
- `oscilationSpeed` - Set speed at which will strength of the wind oscilate between no wind and maximum wind.